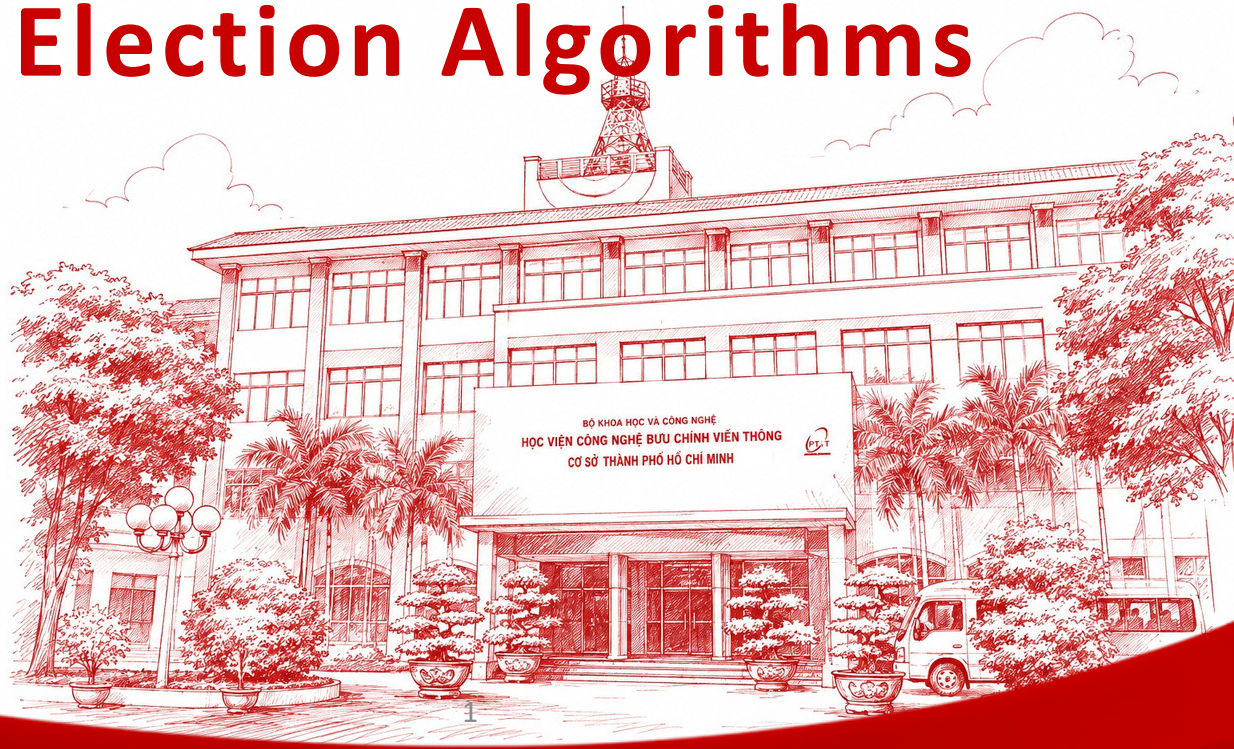
INT1405 - DISTRIBUTED SYSTEMS

LECTURE 04
Processes, Threads
Mutual Exclusion and Election Algorithms

Semester 1, 2026-2027

Lê Hà Thanh
lehathanh@ptithcm.edu.vn



Tanenbaum & van Steen (2nd Edition),
Chapter 3 (3.1–3.5) & Chapter 6 (6.3, 6.5)

Learning Objectives & Roadmap

Core Technical Competencies

- Compare process-level vs. thread-level execution models in terms of context switching overhead.
- Evaluate thread organization strategies in high-concurrency servers (Multithreaded vs Event-driven).
- Classify virtualization models (Hypervisors vs. Containers) and code mobility paradigms.
- Analyze message complexities, synchronization delays, and SPOF vulnerabilities across 4 mutual exclusion algorithms.
- Trace exact message exchanges during coordinator failures using Bully and Ring election algorithms.

Course Curriculum Mapping

- Textbook Reference: Tanenbaum & van Steen (2nd Ed.), Chapter 3 & Chapter 6.
- Prerequisite Knowledge: Lecture 02 (RPC/Messaging) & Lecture 03 (Time/Clocks).
- Practical Component: Multi-threaded election simulator lab assignment.

Content

1. Module 1: Processes, Multithreading & Virtualization
2. Module 2: Server Architectures & Code Migration Frameworks
3. Module 3: Distributed Mutual Exclusion Algorithms
4. Module 4: Distributed Leader Election Protocols
5. Module 5: Case Studies, Assessment & Practical Lab

Process Abstraction vs. Thread-Level Concurrency

Process-Level Abstraction

- Process Definition: An executing program instance with an isolated virtual address space.
- Contains OS security context, page tables, global descriptors, and file handles.
- High Creation Overhead: Allocating kernel data structures and memory mappings.
- High Context Switching Overhead: Requires flushing MMU page tables and TLB caches.
- Process Isolation: Faults in one process do not corrupt memory of other processes.

Thread-Level Concurrency

- Thread Definition: Independent execution path within a shared process address space.
- Shares code segment, global memory, and OS resources with peer threads.
- Maintains private execution context: Registers, Stack Pointer, and Program Counter.
- Low Context Switching Overhead: No MMU page table flushes required.
- Network I/O Concurrency: Prevents single thread blocking from halting entire process.

Threading Architectures in Distributed Clients & Servers

Client-Side Multithreading

- Latency Hiding: Concurrently fetching multi-media page components.
- Parallel Connection Spawning: Spawning multiple HTTP/QUIC socket connections.
- User Interface Responsiveness: Offloading background network I/O from UI threads.

Server-Side Execution Models

- Single-Threaded Server: Sequential processing; blocking socket I/O halts server.
- Multithreaded Server: Dispatcher thread assigns requests to worker thread pool.
- Event-Driven (FSM): Asynchronous non-blocking loop (epoll/select) eliminating context switches.

Role of Virtualization in Distributed Infrastructure

Motivation & Core Drivers

- Hardware Abstraction: Decoupling software execution from physical server hardware.
- Resource Consolidation: Running heterogeneous workloads on multi-core servers.
- Security & Multi-Tenancy: Isolating untrusted tenant workloads in cloud platforms.
- Legacy Application Support: Executing legacy OS software on modern server racks.

Virtualization Interface Levels

- Instruction Set Architecture (ISA): Hardware-level instructions (x86, ARM).
- System Call Interface: Native OS kernel boundaries.
- Application Programming Interfaces (APIs): Middleware / Library calls (JVM, POSIX).

Hypervisor Architectures vs. Process Containers

Hypervisors (Type-1 & Type-2)

- Type-1 (Bare-Metal): Runs directly on hardware (VMware ESXi, Xen, KVM). High efficiency.
- Type-2 (Hosted): Runs on top of host OS (VirtualBox). Higher virtual OS overhead.
- Full Hardware Emulation: Each Virtual Machine executes its own complete Guest OS kernel.

OS-Level Containers

- Shared Kernel Model: Shares host kernel while isolating namespaces and cgroups.
- Lightweight & Fast: Zero guest OS boot delay; spins up in milliseconds (Docker, LXC).
- Near-Native Performance: Minimal memory footprint and hardware emulation overhead.

Stateful vs. Stateless Server Design

Stateless Servers

- No Session Retention: Server holds no client interaction history between requests.
- High Fault Tolerance: Server crash does not destroy client state; clients simply retry.
- Easy Horizontal Scaling: Requests can be routed to any active server replica.

Stateful & Soft-State Servers

- Session State Maintenance: Server maintains active client sessions (shopping carts, locks).
- Complex Crash Recovery: Server failure loses session state unless replicated.
- Soft State Pattern: Session state expires automatically via Time-To-Live (TTL) timers.

Multi-Tier Server Clusters & Load Balancing

3-Tier Cluster Architecture

- Tier 1: Front-end Load Balancer / Switch (Round-Robin DNS, Content Switches).
- Tier 2: Application Server Pool executing core business logic.
- Tier 3: Back-end Enterprise Database and Storage Cluster.

TCP Handoff Mechanism

- Connection Ingestion: Front-end router accepts client TCP connection and inspects payload.
- Handoff Protocol: Front-end hands off active TCP connection to a back-end worker node.
- Direct Response: Worker node returns response directly to client, bypassing switch.

Content

1. Module 1: Processes, Multithreading & Virtualization
2. Module 2: Server Architectures & Code Migration Frameworks
3. Module 3: Distributed Mutual Exclusion Algorithms
4. Module 4: Distributed Leader Election Protocols
5. Module 5: Case Studies, Assessment & Practical Lab

Code Migration Frameworks & Motivation

Motivation & Performance Drivers

- Minimizing Network Traffic: Moving computation closer to massive data repositories.
- Dynamic Load Balancing: Shifting active processes from overloaded nodes to idle nodes.
- Client-Side Flexibility: Shipping executable components (JavaScript, WebAssembly) to clients.

Anatomy of a Running Process

- Code Segment: Static machine instructions comprising the executable program.
- Resource Segment: References to external resources (files, sockets, database links).
- Execution Segment: Runtime state (call stack, registers, local variables, PC).

Mobility Paradigms: Weak vs. Strong Mobility

Weak Mobility

- Code Segment Transfer: Transfers only the executable code segment (and initial parameters).
- Execution Restart: Process always starts execution from scratch at target host.
- Lightweight Implementation: Used in Java Applets, WebAssembly, and remote execution scripts.

Strong Mobility

- Code + Execution Transfer: Transfers code segment AND complete execution stack.
- Instruction Resume: Process resumes from the exact instruction where it was paused.
- Complex Implementation: Requires OS/VM support to capture stack frames and CPU registers.

Resource Binding Strategies During Code Migration

Resource Classifications

- Unattached Resources: Easy to move or recreate (shared data files, memory buffers).
- Fastened Resources: Costly or difficult to move (large localized databases).
- Fixed Resources: Hard-bound to physical host hardware (local devices, GPU accelerators).

Binding Mechanisms

- By Value (Unique): Move resource or set up network proxy.
- By URL / Reference: Re-bind reference or set up network proxy.
- By Type (Generic): Re-bind to local generic resource at destination host.

Resource Binding Action Matrix

Resource Binding Type	Unattached Resource	Fastened Resource	Fixed Resource
By Value (Unique)	Move Resource	Re-bind / Copy	Network Proxy
By URL / Reference	Re-bind / Copy	Re-bind / Copy	Network Proxy
By Type (Generic)	Re-bind Local	Re-bind Local	Re-bind Local

Content

1. Module 1: Processes, Multithreading & Virtualization

2. Module 2: Server Architectures & Code Migration Frameworks

3. Module 3: Distributed Mutual Exclusion Algorithms

4. Module 4: Distributed Leader Election Protocols

5. Module 5: Case Studies, Assessment & Practical Lab

The Distributed Mutual Exclusion Problem

Core Problem & Challenges

- Shared Resource Access: Coordinating access to critical resources without shared memory.
- Asynchronous Networks: Unbounded network packet delays and packet losses.
- Absence of Global Clock: Cannot rely on physical wall-clock timestamps for locking.

4 Evaluation Criteria

- Safety (Mutual Exclusion): At most one process can execute in Critical Section (CS) at a time.
- Liveness (No Deadlock/Starvation): Every requesting process eventually enters CS.
- Ordering: CS requests are serviced in logical causal order (Lamport timestamps).
- Low Overhead: Minimizing message overhead per request/release cycle.

Centralized Lock Coordinator Algorithm

Protocol Sequence

- Request CS: Client sends REQUEST to Central Coordinator. If CS free, Coordinator grants.
- Queueing: If CS busy, Coordinator queues request and defers reply.
- Release CS: Client sends RELEASE to Coordinator; Coordinator pops and grants next.

Performance & Drawbacks

- Low Message Overhead: Exactly 3 messages per CS entry (REQUEST, GRANT, RELEASE).
- Short Delay: 2 message propagation delays before entry.
- Single Point of Failure (SPOF): Coordinator crash halts entire system.

Decentralized Majority Voting Algorithm

Majority Voting Mechanics

- Replicated Coordinators: Replicates lock coordinator across n independent nodes.
- Majority Quorum: Client must secure majority vote ($m > n/2$) of coordinator tokens.
- Denial Handling: If majority not secured, client releases acquired votes and backs off.

Fault Tolerance Properties

- Tolerates Coordinator Crashes: System functions as long as majority quorum remains active.
- No Single Point of Failure: Eliminates master node dependency.
- Starvation Risk: Contention between clients can split votes and cause starvation.

Ricart & Agrawala Distributed Contention Protocol

Protocol Execution

- Multicast Request: Client multicasts $REQUEST(T_i, P_i)$ with Lamport timestamp to all peers.
- Peer Evaluation: Peers reply OK if not in CS and do not want CS.
- Timestamp Priority: If peer also wants CS, it compares timestamps; smaller timestamp wins OK reply.
- Entry Condition: Client enters CS upon receiving OK replies from all $N - 1$ peers.

Properties & Limitations

- Message Overhead: $2(N - 1)$ messages per CS entry.
- Fully Distributed: No central coordinator required.
- Fragile Reliability: Failure of ANY single node halts the entire system.

Token Ring Overlay Protocol

Logical Ring Organization

- Ring Topology Overlay: Nodes ordered logically into unidirectional ring ($P0 \rightarrow P1 \rightarrow \cdots \rightarrow P0$).
- Token Circulation: Single special Token message passes continuously around ring.
- CS Execution: Node holding token enters CS; forwards token to neighbor upon exit.

Key Properties

- Starvation Free: Guarantees fair access; token visits every node sequentially.
- Variable Delay: Entry delay ranges from 0 to $N - 1$ message hops.
- Continuous Overhead: Token circulates continuously even when no node requests CS.

Token Ring Failure Modes & Recovery

Token Loss & Detection

- Token Loss Hazard: Token dropped due to node crash or network error.
- Loss Detection: Nodes monitor token circulation timer threshold $T_{\max}$.
- Regeneration Protocol: Loss triggers token regeneration (risk of duplicate tokens!).

Ring Node Crashes

- Broken Ring Link: Node crash prevents token forwarding.
- Ring Repair: Predecessor node bypasses dead node and updates ring neighbor pointer.

Comparative Analysis of Mutual Exclusion Algorithms

Algorithm	Messages per CS Entry	Delay before CS Entry	Key Vulnerability / Drawback
Centralized	3	2 message delays	Single Point of Failure (Coordinator Crash)
Decentralized	2mk + retries	Variable (Retries)	Starvation under high contention
Distributed (Ricart-Agrawala)	2(N-1)	2(N-1) message delays	Failure of ANY single process halts system
Token Ring	1 to infinity	0 to N-1 hops	Token loss / Ring link failure

System Design Trade-Offs & Selection Rules

Architectural Guidelines

- Centralized Preferred: Stable LAN clusters prioritizing low message count (3 messages).
- Decentralized Preferred: Dynamic P2P networks where no single master node is trusted.
- Token Ring Preferred: Static server farms requiring strict bounded wait times (N-1 hops).

Fundamental Engineering Trade-Off

- No Universal Solution: No algorithm optimizes Safety, Liveness, Low Overhead, and Fault Tolerance simultaneously.
- Modern Cloud Hybrid Approach: Centralized lock managers backed by Raft/Paxos consensus.

Content

1. Module 1: Processes, Multithreading & Virtualization
2. Module 2: Server Architectures & Code Migration Frameworks
3. Module 3: Distributed Mutual Exclusion Algorithms
4. Module 4: Distributed Leader Election Protocols
5. Module 5: Case Studies, Assessment & Practical Lab

Distributed Leader Elections: Objectives & Requirements

Core Objective

- Coordinator Requirement: Distributed architectures require a master node (lock coordinator, primary DB).
- Dynamic Re-Election: When active coordinator fails, cluster must elect a new leader automatically.

System Assumptions

- Unique Priority IDs: Every process P_i has a unique priority identifier number.
- Election Goal: Elect the active process with the HIGHEST priority ID number.
- Global Knowledge: All processes know priority IDs of peer nodes.

The Bully Election Algorithm Protocol

Bully Protocol Sequence

- Failure Detection: Process P_k detects coordinator timeout and transmits ELECTION to higher IDs.
- Higher ID Response: Higher nodes reply OK and take over election; P_k steps back.
- Victory Announcement: If no higher node replies within timeout T, P_k wins and broadcasts COORDINATOR.

Core Characteristic

- Bullying Behavior: Higher-numbered processes forcefully push lower-numbered processes out of the way.

Bully Algorithm Performance & Message Complexity

Best-Case Complexity

- Second-Highest ID Detects Failure: Sends 1 set of ELECTION messages, gets no reply, broadcasts victory.
- Best-Case Message Cost: $O(N)$.

Worst-Case Complexity

- Lowest ID (P_1) Detects Failure First: Every node sequentially triggers elections.
- Worst-Case Message Cost: $O(N^2)$.
- False Election Hazard: Unreliable failure detectors cause unnecessary costly elections.

Ring-Based Leader Election Protocol

Ring Election Mechanics

- Logical Ring Topology: Processes ordered in a unidirectional ring overlay.
- ELECTION Message Circuit: Initiator builds ELECTION message with its ID; forwards around ring.
- ID List Accumulation: Each active node appends its ID to message list before forwarding.
- Victory Announcement: Returning message identifies MAX ID; initiator broadcasts COORDINATOR.

Key Advantage

- Deterministic Message Cost: Exactly $2N$ messages (N to gather IDs, N to announce winner).

Ring Election Complexity & Multiple Initiators

Multiple Initiator Handling

- Simultaneous Triggers: Multiple nodes detecting failure spawn independent ELECTION messages.
- Identical Result: All messages complete circuit and pick the exact same MAX ID leader.
- No Message Explosion: Message overhead remains linear $O(N)$ even with concurrent elections.

Comparison with Bully

- Ring Election: Predictable $2N$ messages; higher latency.
- Bully Election: $O(N^2)$ worst case; lower latency.

Leader Elections in Wireless / Ad-Hoc Networks

Wireless Challenges

- Dynamic Topology: Node mobility, lossy links, and limited transmission range.

Tree-Based Election Protocol

- Query Broadcast: Initiator broadcasts ELECTION query to radio neighbors.
- Spanning Tree Construction: Nodes construct Directed Acyclic Graph (DAG) rooted at initiator.
- Upward Metric Aggregation: Leaf nodes report maximum capacity/ID upward to parents.
- Root Selection: Root node identifies best candidate and broadcasts choice down tree.

Content

1. Module 1: Processes, Multithreading & Virtualization

2. Module 2: Server Architectures & Code Migration Frameworks

3. Module 3: Distributed Mutual Exclusion Algorithms

4. Module 4: Distributed Leader Election Protocols

5. Module 5: Case Studies, Assessment & Practical Lab

Real-World Case Study: Apache ZooKeeper & ZAB

Enterprise Role

- Coordination Kernel: Used across cloud infrastructure (Apache Kafka, Hadoop, HBase).

ZAB Leader Election Protocol

- Fast Leader Election: Candidates vote based on epoch numbers and transaction IDs (zxid).
- Most Up-To-Date Leader: Candidate with highest zxid (most recent transaction log) wins election.
- Consensus Guarantee: Ensures elected leader possesses all committed transactions.

Real-World Case Study: Live Virtual Machine Migration

Enterprise Role

- Zero-Downtime Maintenance: Hypervisor patching and load rebalancing in AWS / VMware.

Execution Steps

- 1. Pre-Copy Phase: Copy memory pages from source to target host while VM executes.
- 2. Delta Sync: Re-transmit dirtied memory pages iteratively.
- 3. Stop-and-Copy Phase: Briefly pause VM (<50ms), transfer CPU execution stack (Strong Mobility).
- 4. Network Re-binding: Update network switch ARP tables to route traffic to target.

Numerical & Analytical Review Problems

Problem 1: Ricart-Agrawala Tie-Breaking

- Scenario: P_1 and P_3 request CS entry simultaneously.
- Timestamps: P_1 sends $(T_1 = 12, P_1)$; P_3 sends $(T_3 = 12, P_3)$.
- Question: Which process enters CS first?
- Solution: Timestamps identical $(T = 12)$. Process ID tie-breaker rule applies: $P_1 < P_3 =>$ P_1 enters first; P_3's reply is deferred by P_1.

Problem 2: Bully Message Complexity

- Scenario: Cluster of N nodes where lowest ID (P_1) detects failure first.
- Question: What is the total message complexity?
- Solution: $O(N^2)$ worst-case message complexity due to cascading elections.

Conceptual Review & Self-Assessment Questions

Question 1: Mobility Paradigms

- Explain why Live VM Migration requires Strong Mobility, whereas browser JavaScript requires only Weak Mobility.

Question 2: Token Ring Failures

- What are the two primary failure modes of Token Ring mutual exclusion, and how does the protocol recover from each?

Practical Programming Lab: Distributed Leader Election

Simulator Specifications

- Multi-Threaded Architecture: Spawn 5 process instances $(P_1 \dots P_5)$ communicating over sockets.
- Protocol Implementation: Implement Bully Algorithm OR Ring Election Protocol.
- Fault Injection: Simulate coordinator crash by terminating active leader thread (P_5).
- Verification: Confirm remaining nodes detect timeout and elect new leader P_4.

Lab Deliverables

- Executable Source Code (Python, Java, or Go).
- Console Trace Output Log showing election message sequences.
- 2-Page Architectural Design Report.

Lab Expected Execution Trace & Evaluation Rubric

Sample Console Trace Log

- [SYSTEM] Active Leader P5 terminated.
- [P2] Timeout detected! Initiating Election...
- [P2] Send ELECTION -> P3, P4, P5
- [P3] Recv ELECTION -> Reply OK
- [P4] Recv ELECTION -> Reply OK
- [P4] Send ELECTION -> P5
- [P4] Timeout waiting for P5 -> NEW LEADER!
- [P4] Broadcast COORDINATOR victory to P1, P2, P3

Grading Rubric

- Election Protocol Logic: 40%
- Automated Failure Detection: 30%
- Execution Trace Output: 20%
- Design Report: 10%

Mutual Exclusion: Centralized vs. Distributed Trade-Offs

Centralized Model Advantages

- Low Message Complexity: 3 messages per CS entry.
- Simple Implementation: Single lock queue.

Distributed Model Advantages

- Eliminates Master Bottleneck: Load distributed across all nodes.
- High Availability Potential: Decentralized voting tolerates node crashes.

Election Protocols: Bully vs. Ring Trade-Offs

Bully Algorithm Characteristics

- Lower Latency: Fast completion if highest node initiates.
- High Message Overhead: $O(N^2)$ in worst-case scenario.

Ring Algorithm Characteristics

- Predictable Overhead: Exactly $2N$ messages always.
- Higher Latency: Sequential ring circulation delays.

Virtualization & Code Mobility Summary

Virtualization Paradigms

- Type-1 Hypervisors: Maximum isolation & performance.
- Containers: Maximum efficiency & fast deployment.

Code Mobility Drivers

- Bandwidth Optimization: Process data locally.
- Dynamic Rebalancing: Migrate active workloads.

Fault Tolerance in Distributed Coordination

Mutual Exclusion Resilience

- Centralized: Requires backup coordinator.
- Token Ring: Requires token loss detection & ring repair.

Election Resilience

- Bully: Sensitive to false failure detections.
- Ring: Requires bypass of dead ring nodes.

Advanced Synchronization & Coordination Primitives

Consensus Protocols

- Paxos & Raft: Solving agreement in untrusted asynchronous networks.
- Atomic Broadcast: Guaranteeing total order delivery across replicas.

Industry Applications

- etcd / Kubernetes: Cluster state management using Raft consensus.
- Chubby / Spanner: Lock services powering cloud infrastructure.

Comprehensive Exam Study Review Checklist

Process & Thread Concepts

- Process vs Thread context switching costs.
- Dispatcher / Worker vs Event-Driven server models.
- Weak vs Strong Code Mobility & Resource Binding.

Coordination Algorithms

- Ricart-Agrawala message costs and timestamp sorting.
- Bully vs Ring election protocols and message bounds.

Lecture 04 Summary & Key Takeaways

Key Takeaways for Lecture 04

- Processes & Threads: Multithreading is essential for hiding network I/O latency in distributed clients and servers.
- Virtualization & Mobility: OS Containers provide lightweight isolation, while Strong Mobility preserves execution stack state during process migration.
- Mutual Exclusion: Centralized algorithms offer low message complexity (3 msgs) but suffer SPOF; Distributed algorithms (Ricart-Agrawala) eliminate masters but increase message overhead $(2(N-1)$ msgs).
- Leader Elections: Bully algorithm offers fast execution at $O(N^2)$ message cost; Ring algorithm guarantees linear $2N$ message cost.
- Industrial Practice: Production cloud platforms (ZooKeeper, Kubernetes) combine centralized lock servers with Raft/Paxos consensus for high availability.